

UNIVERSIDAD / INSTITUCIÓN

Informe de Proyecto de Software

ULTIMATE EN TRON

Videojuego 2D en C++17 y Qt 6

Juan Diego Arias Toro

Aníbal José Guerra Soler

Informática II

Tabla de contenido

No se encuentran entradas de índice.

1. Introducción
2. Planteamiento del problema
3. Definición general y objetivos
4. Especificación de requerimientos
5. Metodología y planificación
6. Diseño y arquitectura del sistema
7. Diagrama general de clases
8. Desarrollo e implementación
9. Procedimientos de prueba
10. Resultados y discusión
11. Conclusiones
12. Referencias

1. Introducción

Ultimate en TRON es un videojuego 2D desarrollado en C++17 con Qt 6, concebido como un proyecto académico de software interactivo que integra arquitectura orientada a objetos, renderizado bidimensional, manejo de eventos, temporización, gestión de recursos audiovisuales y lógica de juego. La propuesta adopta una estética neón/cyber inspirada en entornos digitales de tipo *TRON* y se estructura como una aplicación local con dos niveles jugables, sprites, audio y estados de victoria o derrota.

El propósito del proyecto fue demostrar, en un mismo sistema, la integración de conceptos fundamentales del curso, entre ellos programación orientada a objetos, separación por capas, uso de memoria dinámica, contenedores STL, físicas visibles, diseño de interfaz gráfica en Qt e incorporación de un agente inteligente sencillo. El público objetivo del proyecto incluye estudiantes, docentes, evaluadores de proyectos de software y usuarios interesados en observar un prototipo jugable de precisión y defensa con una base técnica justificable.

2. Planteamiento del problema

El problema abordado en este proyecto consiste en desarrollar un software interactivo que no se limite a mostrar una interfaz o una animación aislada, sino que integre de manera coherente jugabilidad, retroalimentación visual y sonora, reglas de interacción, estados del sistema y una arquitectura técnicamente defendible. En el contexto académico del curso, el desafío no era únicamente construir un videojuego funcional, sino diseñar una solución que permitiera justificar decisiones de modelado, herencia, memoria dinámica, contenedores, físicas, GUI y comportamiento autónomo.

Muchos prototipos académicos se reducen a interacciones simples o a demostraciones parciales de conceptos. En contraste, este proyecto busca articular una experiencia completa en la que el usuario pueda verificar cómo reaccionan las entidades del sistema ante entradas, temporizadores, colisiones, objetivos y condiciones de victoria o derrota. En este sentido, el videojuego se convierte en un escenario adecuado para evidenciar la relación entre modelo, lógica, interacción y representación gráfica.

3. Definición general y objetivos

El sistema propuesto se compone de dos modos principales. El **Nivel 1, Ruta de Transmisión**, exige lanzar un disco por trazado hacia checkpoints obligatorios, aprovechando superficies de rebote y evitando drones y obstáculos. El **Nivel 2, Defensa del Núcleo**, presenta una defensa frontal en perspectiva, en la que el jugador debe interceptar enemigos entrantes antes de que alcancen la zona protegida. Esta diferenciación entre niveles busca cumplir no solo un objetivo jugable, sino también una

exigencia de diseño y arquitectura, al demostrar que ambos escenarios responden a lógicas distintas.

Objetivo general

Desarrollar un videojuego 2D funcional, modular y técnicamente sustentable, con dos niveles jugables, recursos audiovisuales, retroalimentación visual y una integración sencilla pero real de un agente inteligente.

Objetivos específicos

- Implementar una capa de lógica independiente de Qt.
- Construir una interfaz en Qt para renderizado, manejo de entrada, audio y HUD.
- Definir entidades como jugador, disco, drones, checkpoints, obstáculos y discos enemigos.
- Implementar estados de victoria, derrota y reinicio.
- Construir una vista visual tipo tercera persona para el Nivel 2.
- Integrar un agente inteligente persistente en el Nivel 1.
- Incorporar sprites, audio y temporización dentro del flujo general del juego.

4. Especificación de requerimientos

4.1 Requisitos funcionales

El sistema debe permitir iniciar la aplicación y desplegar un menú principal, seleccionar el Nivel 1 o el Nivel 2, reiniciar el nivel activo y regresar al menú. En el Nivel 1, el jugador debe poder lanzar el disco mediante interacción con el mouse, validar checkpoints en orden, materializarse en el último checkpoint alcanzado, reiniciar desde dicho punto ante colisión, aplicar rebotes y completar el recorrido al llegar al núcleo. En el Nivel 2, el sistema debe generar discos enemigos, permitir acciones defensivas, destruir amenazas y declarar condiciones de victoria o derrota. En ambos niveles, deben renderizarse sprites, HUD, efectos visuales y audio.

4.2 Requisitos no funcionales

El proyecto debe mantener separación estricta entre la capa logic y la capa gui, compilar con **C++17** y **Qt 6**, ofrecer un rendimiento suficiente para actualización fluida, manejar fallback visual cuando algún sprite no carga, conservar mantenibilidad mediante clases con responsabilidades claras y ejecutarse localmente en Windows con Qt y MinGW. Además, la solución debe evitar dependencias externas innecesarias y compilar mediante CMake.

4.3 Restricciones

La capa lógica no debe depender de Qt. Los ajustes realizados en un nivel no deben comprometer la estabilidad del otro. No se emplean librerías externas de inteligencia

artificial ni motores de física o render 3D. Los recursos visuales y sonoros se cargan desde el sistema de recursos QRC. La compilación del proyecto se realiza mediante CMake.

4.4 Usuarios destinatarios

Los usuarios destinatarios del proyecto son el jugador casual, el docente evaluador, el equipo de desarrollo y estudiantes interesados en sustentar la arquitectura y la lógica de un videojuego 2D con base técnica clara.

5. Metodología y planificación

La metodología empleada fue de carácter **iterativo e incremental**, próxima a un enfoque ágil. El proyecto evolucionó mediante ciclos cortos de implementación, compilación, prueba manual, ajuste localizado y nueva validación. Esta forma de trabajo permitió corregir progresivamente problemas de arquitectura, jugabilidad, integración visual y comportamiento de entidades sin perder control sobre el estado general del sistema.

Las fases principales del desarrollo fueron las siguientes: estructuración inicial de la base del proyecto, modelado de entidades lógicas, implementación funcional del Nivel 1, ajuste geométrico y jugable del mismo, construcción del Nivel 2, integración de sprites, audio y HUD, mejora visual hacia una perspectiva más marcada en tercera persona, incorporación del agente inteligente y etapa final de pruebas y documentación. El cronograma estimado se distribuyó en siete semanas, cubriendo análisis, arquitectura, desarrollo de niveles, integración audiovisual, IA simple, balance y cierre documental.

6. Diseño y arquitectura del sistema

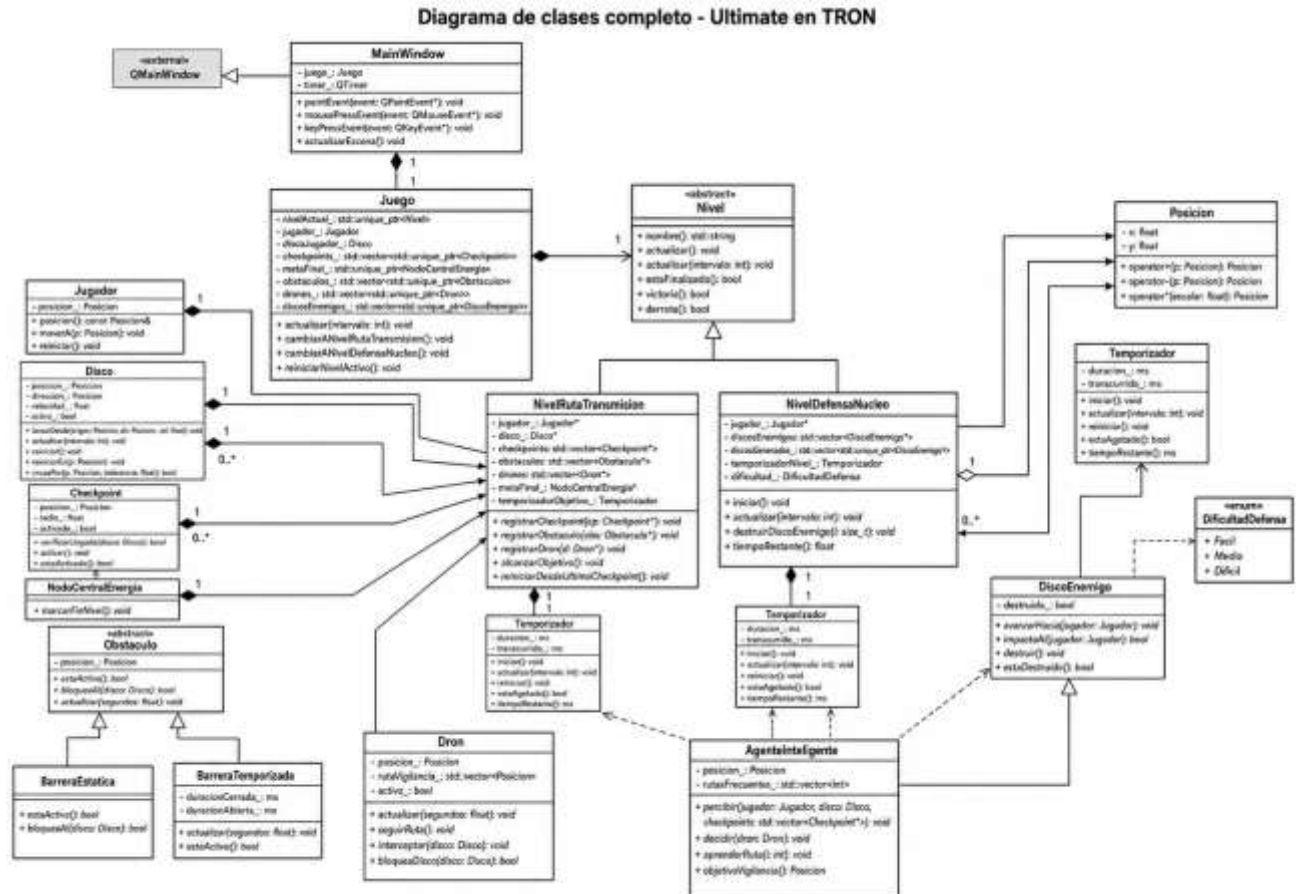
La arquitectura del sistema se divide en dos capas principales: **logic** y **gui**. La capa logic contiene entidades, niveles, reglas del juego, temporizadores y estados, sin dependencia directa de Qt. La capa gui, por su parte, contiene la clase `MainWindow`, la conversión de coordenadas, el render con `QPainter`, la carga de sprites, el sonido y el manejo de eventos de teclado y mouse. Esta separación fue una decisión central de diseño, ya que permitió modificar elementos visuales y de interacción sin alterar las reglas internas del sistema.

La jerarquía general del sistema puede resumirse como:

MainWindow → **Juego** → **Nivel** → {**NivelRutaTransmision**, **NivelDefensaNucleo**}.

La clase `Juego` coordina las entidades principales, entre ellas `Jugador`, `Disco`, `Checkpoint`, `Dron`, `Obstaculo` y `NodoCentralEnergia`. `NivelRutaTransmision` integra al `AgenteInteligente`, mientras que `NivelDefensaNucleo` administra enemigos, temporización y dificultad. Las dependencias externas del sistema incluyen `Qt6::Widgets`, `Qt6::Multimedia`, la STL de C++ y

7. Diagrama general de clases



En términos generales, la arquitectura de clases se organiza alrededor de tres ideas principales:

1. una capa de presentación representada por `MainWindow`;
2. una clase coordinadora (`Juego`) responsable del estado global;
3. y un conjunto de entidades y niveles especializados que encapsulan la lógica del juego.

La versión completa del diagrama muestra además relaciones de herencia, composición, uso de memoria dinámica, dependencia del agente inteligente y el papel de estructuras de apoyo como Posicion, Temporizador y la enumeración DificultadDefensa. El diagrama incluido en el informe corresponde a la versión final del diseño.

8. Desarrollo e implementación

La lógica del juego se basa en **actualizaciones temporales**. En cada ciclo se calcula el tiempo transcurrido a partir del intervalo y se actualizan entidades móviles, temporizadores, colisiones y estados del sistema. El disco mantiene posición, posición anterior, dirección, velocidad y estado activo o en movimiento para permitir detección por cruce de trayectoria y control del flujo de juego.

En el **Nivel 1**, los checkpoints deben recorrerse en orden. Al alcanzar uno de ellos, este se activa, el jugador se materializa en esa posición y el disco se reinicia desde allí. Las superficies de rebote modifican la trayectoria del disco mediante reflexión, mientras que los drones patrullan rutas o reciben objetivos limitados según el estado del entorno. El progreso del jugador depende de una combinación entre precisión, lectura del entorno y aprovechamiento del tiempo disponible.

El **Agente Inteligente** percibe la posición del jugador, el disco, los checkpoints y el objetivo actual. A partir de esa información registra rutas frecuentes en eventos de llegada o reinicio, y actúa de forma controlada sobre el defensor rojo final. El comportamiento fue ajustado para evitar que el dron bloqueara injustamente checkpoints intermedios, preservando así la jugabilidad del nivel.

En el **Nivel 2**, los enemigos avanzan hacia el jugador de acuerdo con la dificultad seleccionada. El disparo defensivo utiliza una dirección normalizada y un mecanismo de *cooldown* para regular la frecuencia de disparo. Los enemigos destruidos se filtran del render, mientras que la vista visual se refuerza mediante un corredor en perspectiva, escalado por profundidad y sprites rojos para las amenazas entrantes. Con ello se buscó una diferenciación clara frente a la vista cenital del primer nivel.

8.1 Modelos físicos implementados

El proyecto incorpora varias dinámicas visibles; entre las tres principales se destacan las siguientes:

Rebote del disco.

Las superficies de rebote alteran la trayectoria del disco mediante una reflexión respecto a la normal de la superficie. Esta mecánica es central en el Nivel 1 y condiciona la forma de alcanzar checkpoints avanzados.

Compuertas y barreras temporizadas.

Las compuertas se abren y cierran periódicamente, generando ventanas de paso. Este comportamiento obliga al jugador a sincronizar el disparo con el estado del entorno.

Patrullaje de drones.

Los drones siguen trayectorias verticales u horizontales y generan bloqueo espacial. Su movimiento introduce presión temporal y obliga a predecir trayectorias seguras.

A estas dinámicas se suman la temporización de objetivos, el movimiento del disco y la detección de colisiones o intercepciones, que complementan el comportamiento general del sistema.

8.2 Memoria dinámica, STL y excepciones

La gestión de memoria se apoya en `std::unique_ptr` y contenedores STL, principalmente `std::vector`, con el fin de expresar ownership claro de las entidades principales. La clase Juego actúa como propietaria de niveles, obstáculos, drones, checkpoints y discos enemigos, mientras que los niveles usan punteros no propietarios para referenciar entidades administradas externamente.

El proyecto también hace uso de excepciones en casos de configuración o invariantes rotas, evitando utilizarlas como mecanismo de control del gameplay normal. Esto favorece una arquitectura más robusta y facilita la depuración de estados inválidos.

9. Procedimientos de prueba

Caso de prueba	Objetivo	Procedimiento	Resultado esperado	Resultado obtenido
CP-01 Inicio de la aplicación	Verificar que el ejecutable inicia correctamente	Ejecutar la aplicación desde el entorno de desarrollo o desde el ejecutable generado	La ventana principal se abre y muestra el menú inicial sin errores	La aplicación inicia correctamente y despliega el menú principal
CP-02 Selección de Nivel 1	Verificar acceso funcional al Nivel 1	Desde el menú, seleccionar la opción correspondiente al Nivel 1	El sistema cambia al Nivel 1 y muestra HUD, sprites y controles asociados	El Nivel 1 carga correctamente y permite iniciar la partida
CP-03 Lanzamiento por trazado	Verificar la mecánica de disparo mediante mouse	Hacer clic, arrastrar y soltar para definir trayectoria del disco	El disco se lanza en la dirección indicada y entra en movimiento	La trayectoria del disco responde al trazado realizado por el usuario
CP-04 Alcance de checkpoint	Validar activación de checkpoints en orden	Completar el recorrido hasta el checkpoint objetivo	El checkpoint se activa, el jugador se materializa en la nueva posición y el disco se reinicia allí	El sistema actualiza correctamente el progreso del jugador

Caso de prueba	Objetivo	Procedimiento	Resultado esperado	Resultado obtenido
CP-05 Colisión con dron en Nivel 1	Verificar reinicio por colisión	Lanzar el disco de manera que impacte un dron	El nivel reinicia desde el último checkpoint válido	El reinicio ocurre correctamente desde el último checkpoint alcanzado
CP-06 Rebote del disco	Verificar el comportamiento de superficies de rebote	Lanzar el disco contra una superficie rosada	La trayectoria del disco cambia según la superficie de rebote	El rebote ocurre y permite continuar el recorrido del nivel
CP-07 Finalización del Nivel 1	Verificar condición de victoria del Nivel 1	Completar todos los checkpoints y llegar al núcleo final	El sistema declara victoria y muestra el overlay correspondiente	El nivel finaliza correctamente con estado de victoria
CP-08 Selección de Nivel 2	Verificar acceso funcional al Nivel 2	Desde el menú, seleccionar la opción correspondiente al Nivel 2	El sistema cambia al Nivel 2 y muestra la escena defensiva	El Nivel 2 carga correctamente con su vista diferenciada
CP-09 Generación de enemigos	Verificar aparición de discos enemigos	Iniciar el Nivel 2 y esperar el ciclo normal de juego	Los enemigos aparecen progresivamente según la lógica del nivel y la dificultad	Los discos enemigos se generan y avanzan correctamente
CP-10 Destrucción de enemigos	Verificar eliminación de amenazas	Disparar al enemigo y acertar con el proyectil defensor	El enemigo es destruido y deja de renderizarse en pantalla	El disco enemigo desaparece correctamente tras la intercepción
CP-11 Victoria en Nivel 2	Verificar condición de victoria del Nivel 2	Resistir hasta agotar el tiempo o la condición establecida	El sistema declara victoria y muestra el overlay correspondiente	La partida finaliza correctamente con victoria
CP-12 Derrota en Nivel 2	Verificar condición de derrota del Nivel 2	Permitir que una amenaza alcance al jugador o condición equivalente	El sistema declara derrota y muestra el overlay correspondiente	La derrota se activa correctamente
CP-13 Cambio de dificultad	Validar la dificultad configurable	Seleccionar Fácil, Medio y Difícil desde el menú antes de iniciar el Nivel 2	El comportamiento del nivel cambia según la dificultad escogida	Se observan cambios reales en presión, frecuencia o exigencia del nivel

Caso de prueba	Objetivo	Procedimiento	Resultado esperado	Resultado obtenido
CP-14 Reinicio manual	Verificar reinicio por teclado	Presionar la tecla R durante cualquiera de los niveles	El nivel activo se reinicia correctamente	El reinicio manual funciona sin errores
CP-15 Regreso al menú	Verificar navegación global	Presionar ESC o M durante un nivel	El sistema regresa al menú principal	La transición al menú funciona correctamente
CP-16 Sonido de eventos	Verificar integración de audio	Activar situaciones como checkpoint, destrucción, victoria y derrota	Se reproducen los sonidos correspondientes sin afectar la ejecución	El audio funciona de forma correcta en eventos principales
CP-17 Carga de sprites	Verificar visualización de recursos gráficos	Ejecutar el juego y recorrer ambos niveles	Los sprites se cargan y renderizan correctamente	Los recursos gráficos se muestran de manera estable
CP-18 Estabilidad general	Verificar robustez básica del sistema	Jugar varias partidas, reiniciar niveles y cambiar entre menús	El proyecto se mantiene estable y no presenta cierres inesperados	El sistema compila y ejecuta de forma jugable y estable

En términos generales, los resultados obtenidos fueron satisfactorios. Los casos de prueba ejecutados permitieron verificar la funcionalidad principal de ambos niveles, la respuesta de la interfaz, la integración de recursos audiovisuales y la estabilidad básica del sistema durante la ejecución.

10. Resultados y discusión

El proyecto permitió obtener dos niveles diferenciados: uno centrado en ruta y precisión, y otro en defensa frontal con perspectiva simulada. Además, se integraron sprites, audio, HUD, temporizadores, estados de victoria y derrota, dificultad configurable y un agente inteligente sencillo. Desde el punto de vista académico, el sistema consigue articular arquitectura, lógica de juego e interacción en una única solución defendible.

Entre las limitaciones actuales se encuentran la ausencia de pruebas unitarias automatizadas, la falta de persistencia del aprendizaje del agente entre ejecuciones, el carácter simulado de la perspectiva del Nivel 2, el recorte manual de sprites y el hecho de que parte del balance jugable depende todavía de prueba manual. A pesar de ello, el sistema alcanzó un estado estable, funcional y visualmente coherente para el propósito del proyecto.

Las principales dificultades enfrentadas durante el desarrollo fueron el ajuste de trayectorias de rebote sin comprometer la jugabilidad, la necesidad de diferenciar visualmente el Nivel 2 respecto al Nivel 1, la corrección de sprites sin alterar la lógica interna y la integración del agente inteligente de forma que resultara técnicamente justificable sin volver injusto el sistema para el jugador.

10.1 Cambios respecto a la planeación inicial

Durante el desarrollo, la solución evolucionó de forma significativa respecto a la planeación inicial. El Nivel 1 pasó de una idea general de recorrido a una ruta estructurada con checkpoints obligatorios, compuertas, superficies de rebote y un defensor final reactivo. El Nivel 2, que originalmente se concebía como una defensa más simple, fue reformulado hacia una presentación visual en perspectiva, con enemigos que escalan por profundidad y una sensación más cercana a tercera persona.

Asimismo, la incorporación de sprites y sonido se volvió más amplia de lo previsto inicialmente, y el agente inteligente pasó de ser una idea conceptual a una integración real dentro del flujo del Nivel 1. De igual manera, el diseño de clases evolucionó desde una estructura más reducida hacia una representación completa que incluye capa GUI, lógica, temporización, dificultad y relaciones de dependencia bien definidas.

11. Conclusiones

El proyecto permitió obtener dos niveles diferenciados: uno centrado en ruta y precisión, y otro en defensa frontal con perspectiva simulada. Además, se integraron sprites, audio, HUD, temporizadores, estados de victoria y derrota, dificultad configurable y un agente inteligente sencillo. Desde el punto de vista académico, el sistema consigue articular arquitectura, lógica de juego e interacción en una única solución defendible.

Entre las limitaciones actuales se encuentran la ausencia de pruebas unitarias automatizadas, la falta de persistencia del aprendizaje del agente entre ejecuciones, el carácter simulado de la perspectiva del Nivel 2, el recorte manual de sprites y el hecho de que parte del balance jugable depende todavía de prueba manual. A pesar de ello, el sistema alcanzó un estado estable, funcional y visualmente coherente para el propósito del proyecto.

Las principales dificultades enfrentadas durante el desarrollo fueron el ajuste de trayectorias de rebote sin comprometer la jugabilidad, la necesidad de diferenciar visualmente el Nivel 2 respecto al Nivel 1, la corrección de sprites sin alterar la lógica interna y la integración del agente inteligente de forma que resultara técnicamente justificable sin volver injusto el sistema para el jugador.

10.1 Cambios respecto a la planeación inicial

Durante el desarrollo, la solución evolucionó de forma significativa respecto a la planeación inicial. El Nivel 1 pasó de una idea general de recorrido a una ruta estructurada con checkpoints obligatorios, compuertas, superficies de rebote y un defensor final reactivo. El Nivel 2, que originalmente se concebía como una defensa más simple, fue reformulado hacia una presentación visual en perspectiva, con enemigos que escalan por profundidad y una sensación más cercana a tercera persona.

Asimismo, la incorporación de sprites y sonido se volvió más amplia de lo previsto inicialmente, y el agente inteligente pasó de ser una idea conceptual a una integración real dentro del flujo del Nivel 1. De igual manera, el diseño de clases evolucionó desde una estructura más reducida hacia una representación completa que incluye capa GUI, lógica, temporización, dificultad y relaciones de dependencia bien definidas.

12. Referencias

- Qt Documentation. *Qt Widgets, QPainter, QPixmap y QSoundEffect*. Disponible en: Qt Documentation.
- CMake Documentation. Disponible en: CMake Documentation.
- C++ Reference. *chrono, vector, memory, algorithm*. Disponible en: cppreference.
- Documentación interna del proyecto: `README.md`, `CMakeLists.txt`, `include/logic`, `src/logic`, `include/gui`, `src/gui` y `resources/audio.qrc`